THE ISLAMIA UNIVERSITY OF BAHAWALPUR

Department of Computer Science

Bachelor of Science in Computer Science

Software Design Document (SDD)

AI-Powered Exam Paper Generator System

Field	Details
Project Title	AI-Powered Exam Paper Generator
Document Type	Software Design Document (SDD)
Version	1.0
Date	April 2025
Prepared By	Mujeeb ur Rehman (21886003) | Madad Abbas (21886010)
Semester	7th Semester, BSCS
Supervisor	Syeda Asma Bibi
University	The Islamia University of Bahawalpur

1. Introduction

1.1 Purpose

This Software Design Document (SDD) describes the architectural and detailed design of the AI-Powered Exam Paper Generator System. It translates the requirements specified in the SRS into a concrete technical blueprint that guides the implementation phase of the Final Year Project.

This document covers the system architecture, component design, database schema, API design, AI pipeline design, and interface design.

1.2 Scope

This document covers the complete technical design of all system components including the frontend UI, FastAPI backend, OCR pipeline, RAG engine, LLM integration layer, export module, and question bank database.

1.3 Design Goals

- Modularity — Each system component must be independently replaceable (e.g., swap LLaMA 3 for Mistral without changing other modules)
- Offline-first — The system must function completely without internet access
- Accuracy — Questions generated must be grounded strictly in uploaded content (no hallucination)
- Extensibility — The system must support adding new question types or export formats in future
- Performance — Response time for question generation must stay within acceptable bounds for educator workflows

2. System Architecture

2.1 Architectural Pattern

The system follows a Three-Tier Architecture combined with a Pipeline Architecture for the AI processing layer:

- Tier 1 — Presentation Layer: Browser-based frontend (React or HTML/CSS/JS)
- Tier 2 — Application Layer: FastAPI backend with modular service components
- Tier 3 — Data Layer: ChromaDB (vector store), SQLite (question bank), local file system

Within Tier 2, the AI processing follows a Pipeline Architecture where each stage feeds into the next sequentially:

Upload → OCR/Extraction → Cleaning → Chunking → Embedding → RAG Retrieval → LLM Generation → Validation → Formatting → Export

2.2 High-Level Component Diagram

The system consists of the following major components:

Component	Technology	Responsibility
Frontend UI	HTML+CSS+JS	User interaction, file upload, parameter selection, paper preview
API Gateway	FastAPI (Python)	Route management, request validation, session handling
File Handler	Python (aiofiles, shutil)	Secure file upload, validation, temporary storage, cleanup
OCR Service	PyMuPDF + Tesseract	Text extraction from PDFs and images
Text Processor	Python (re, nltk)	Text cleaning, normalization, chunking
Vector Store Service	ChromaDB + Sentence Transformers	Embedding generation and semantic search (RAG)
LLM Service	Ollama REST API	Question generation via LLaMA 3 8B
Validator	Python (json, pydantic)	Validate LLM output structure and content quality
Deduplicator	scikit-learn (cosine similarity)	Remove near-duplicate questions
Bloom Classifier	Rule-based + LLM-assisted	Map questions to Bloom's Taxonomy levels
Export Service	python-docx + WeasyPrint	Generate DOCX and PDF exam papers
Question Bank	SQLite + SQLAlchemy	Persist and retrieve generated questions

2.3 Deployment Architecture

The entire system runs on a single local machine (no cloud required):

Layer	Host	Port
Frontend (Dev)	Vite / Live Server (Windows)	5173 / 3000
FastAPI Backend	Uvicorn ASGI Server (Windows)	8000
Ollama LLM Server	Ollama for Windows (daemon)	11434
ChromaDB	In-process (embedded mode)	N/A
SQLite	Local file (exam_bank.db)	N/A

3. Module Design

3.1 File Handler Module

Responsibilities

- Accept multipart file uploads via POST /api/upload
- Validate file type (PDF, JPG, PNG only) and file size (max 50 MB)
- Generate a unique session ID (UUID4) per upload batch
- Store files in /tmp/sessions/{session_id}/ directory
- Schedule auto-deletion after 24 hours using a background task

Key Functions

Function	Input	Output	Description
validate_file()	UploadFile	bool	Check MIME type and file size
save_file()	UploadFile, session_id	str (path)	Save file to session directory
cleanup_session()	session_id	None	Delete session directory and all files
create_session()	None	str (UUID)	Generate unique session identifier

3.2 OCR Service Module

Responsibilities

- Detect if a PDF page contains native text or is scanned
- Extract native text using PyMuPDF for digital PDFs
- Apply Tesseract OCR for scanned pages and image files
- Preprocess images to improve OCR accuracy
- Return combined raw text with page metadata

Text Extraction Decision Logic

The module uses a confidence threshold to decide extraction method:

```
if len(native_text.strip()) > MIN_CHARS_THRESHOLD:
    return native_text  # use PyMuPDF result
else:
    return run_tesseract_ocr(page_image)  # fall back to OCR
```

Image Preprocessing Pipeline

- Convert to grayscale
- Apply Otsu's binarization (adaptive thresholding)
- Apply median blur for noise reduction (kernel size 3)

- Upscale to 300 DPI if resolution is below threshold

Function	Input	Output	Description
extract_from_pdf()	file_path	str	Extract text from all pages using hybrid approach
extract_from_image()	file_path	str	Preprocess and OCR single image
preprocess_image()	cv2 image	cv2 image	Apply grayscale, threshold, blur
merge_extractions()	List[str]	str	Combine text from all uploaded files

3.3 Text Processor Module

Responsibilities

- Remove OCR artifacts, headers, footers, and page numbers
- Fix broken hyphenations and line wraps
- Normalize whitespace and Unicode characters
- Split cleaned text into overlapping semantic chunks for RAG

Chunking Strategy

Uses LangChain's RecursiveCharacterTextSplitter with the following parameters:

```
chunk_size = 500        # characters per chunk
chunk_overlap = 50      # overlap between consecutive chunks
separators = ['\n\n', '\n', '.', ' ']  # priority split order
```

Overlap ensures that concepts spanning chunk boundaries are not lost during retrieval.

3.4 Vector Store Service (RAG Engine)

Responsibilities

- Generate semantic vector embeddings for each text chunk
- Store embeddings in ChromaDB with session-scoped collections
- Retrieve the top-N most semantically relevant chunks at generation time
- Clean up the collection after the session ends

Embedding Model

Model: all-MiniLM-L6-v2 (Sentence Transformers)

- Size: 80 MB — runs efficiently on CPU
- Output: 384-dimensional dense vector
- Speed: ~1000 sentences/second on CPU

RAG Retrieval at Generation Time

```
def get_context(question_type: str, difficulty: str, n: int = 5):
    query = f'{question_type} question about {difficulty} level concepts'
    embedding = embed_model.encode([query])
    results = collection.query(query_embeddings=embedding, n_results=n)
    return '\n'.join(results['documents'][0])
```

3.5 LLM Service Module

Responsibilities

- Send structured prompts to Ollama API (localhost:11434)
- Handle streaming and non-streaming responses
- Implement retry logic for failed or malformed responses (max 3 retries)
- Parse and validate JSON output from LLM

Prompt Templates

Three separate prompt templates are maintained for each question type:

Question Type	Prompt Strategy	Output Format
MCQ	Role: Expert examiner. Provide context + count + difficulty. Enforce JSON.	{"mcqs": [{question, options:{A,B,C,D}, answer, explanation, bloom_level}]}
Short Questions	Role: Examiner. Instruct to generate concise questions with model answers.	{"short_questions": [{question, model_answer, bloom_level}]}
Long Questions	Role: Examiner. Request essay-type questions with marking schemes.	{"long_questions": [{question, marks, key_points[], bloom_level}]}

Ollama API Call

```
POST http://localhost:11434/api/generate
{
  "model": "llama3:8b-instruct-q4_K_M",
  "prompt": "<filled prompt template>",
  "stream": false,
  "options": {"temperature": 0.3, "top_p": 0.9, "num_ctx": 4096}
}
```

Temperature 0.3 is chosen deliberately — low enough to keep questions factual and grounded, high enough to avoid repetition.

3.6 Validator and Deduplicator Module

Validation Steps

- Parse LLM output as JSON (strip markdown fences if present)
- Validate against expected Pydantic schema for each question type
- Check that MCQ options are exactly 4 and answer is in {A, B, C, D}
- Check minimum question length (reject questions under 10 characters)
- If validation fails, trigger retry with modified prompt

Deduplication

After all questions are generated, cosine similarity is computed between all question pairs:

```
from sklearn.metrics.pairwise import cosine_similarity
from sentence_transformers import SentenceTransformer

embeddings = model.encode(all_questions)
sim_matrix = cosine_similarity(embeddings)
# Remove question if similarity > 0.85 with any earlier question
```

3.7 Bloom's Taxonomy Classifier

Questions are mapped to six Bloom's levels based on difficulty input and keyword heuristics:

Difficulty	Bloom Levels Targeted	Trigger Keywords
Easy	Remember, Understand	define, list, state, identify, recall, what is
Medium	Apply, Analyze	explain, compare, calculate, demonstrate, solve
Hard	Evaluate, Create	critique, justify, design, construct, evaluate, argue

For more precise classification, a secondary LLM call is made with the generated question asking: 'Classify this question into one of six Bloom's Taxonomy levels. Respond with only the level name.' This is run asynchronously to avoid slowing the main generation pipeline.

3.8 Export Service Module

DOCX Generation

- Uses python-docx with a pre-designed template structure
- Includes university name, subject, total marks, time, and date fields
- Sections: Instructions, MCQs, Short Questions, Long Questions
- MCQ options formatted with A) B) C) D) labels
- Answer key included as a separate hidden section or appendix

PDF Generation

- HTML template rendered with Jinja2, then converted to PDF using WeasyPrint
- Consistent styling with CSS for professional exam paper appearance
- Page numbers and university header/footer included automatically

4. Database Design

4.1 SQLite Schema — Question Bank

Table: questions

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique question identifier
session_id	TEXT	NOT NULL	Session that generated this question
question_text	TEXT	NOT NULL	The question content
question_type	TEXT	NOT NULL	MCQ / Short / Long
difficulty	TEXT	NOT NULL	Easy / Medium / Hard
bloom_level	TEXT		Bloom's Taxonomy level
options	TEXT		JSON string (MCQ options only)
answer	TEXT		Correct answer or model answer
explanation	TEXT		Explanation for MCQ answers
source_topic	TEXT		Topic chunk from which question was derived
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Generation timestamp

Table: sessions

Column	Type	Constraints	Description
session_id	TEXT	PRIMARY KEY	Unique session UUID
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Session creation time
file_count	INTEGER		Number of uploaded files
status	TEXT		active / completed / expired
paper_config	TEXT		JSON string of generation parameters

5. API Design

5.1 REST API Endpoints

Method	Endpoint	Description	Request Body / Params
POST	/api/upload	Upload PDF or image files	multipart/form-data: files[]
GET	/api/session/{id}	Get session status and file list	Path: session_id
POST	/api/generate	Trigger question generation	JSON: session_id, mcq_count, short_count, long_count, difficulty
GET	/api/status/{job_id}	Poll generation progress	Path: job_id
GET	/api/result/{job_id}	Retrieve generated questions	Path: job_id
POST	/api/export/docx	Export exam paper as DOCX	JSON: questions[], paper_meta{}
POST	/api/export/pdf	Export exam paper as PDF	JSON: questions[], paper_meta{}
GET	/api/bank/questions	List all saved questions	Query: type, difficulty, limit
DELETE	/api/bank/{question_id}	Delete a question from bank	Path: question_id
DELETE	/api/session/{id}	Delete session and clean up files	Path: session_id

5.2 Sample Request/Response — /api/generate

Request

```
POST /api/generate
Content-Type: application/json
{
  "session_id": "a3f7c1d2-...",
  "mcq_count": 10,
  "short_count": 5,
  "long_count": 2,
  "difficulty": "medium"
}
```

Response

```
HTTP 202 Accepted
{
  "job_id": "gen-9b4e2a...",
  "status": "processing",
  "estimated_time_seconds": 90
}
```

6. Frontend Design

6.1 Page Structure

Page / Component	Route	Description
Home / Upload Page	/	File upload area, drag-and-drop, session initialization
Configure Page	/configure	Set MCQ count, short/long question count, difficulty
Generating Page	/generating	Real-time progress bar and status messages
Preview Page	/preview	Display generated questions, allow editing before download
Download Page	/download	DOCX and PDF download buttons, topic coverage chart
Question Bank	/bank	Browse, search, and manage saved questions

6.2 Frontend-Backend Communication

- File uploads use FormData with the native Fetch API via multipart POST
- Generation status is polled every 3 seconds using GET /api/status/{job_id}
- All API responses handled with try-catch blocks inside async functions
- Session ID stored in browser sessionStorage (cleared on tab close)

7. Security Design

Threat	Mitigation Strategy
Malicious file upload (e.g., executable disguised as PDF)	Strict MIME type checking + magic byte validation on server side
Path traversal in file access	All files accessed through session-scoped UUID paths only
LLM prompt injection via uploaded content	Content is chunked and embedded — raw text is never directly interpolated into prompts without sanitization
Excessive resource consumption	File size limit (50 MB), rate limiting on /api/generate (5 requests/minute/session)
Session data leakage	Sessions are isolated by UUID; no cross-session file access possible
Data retention violation	Auto-deletion after 24 hours enforced via APScheduler background job

8. Error Handling Strategy

Error Scenario	Detection	Response
File upload failure	Exception in file write	HTTP 400 with descriptive message
OCR returns empty text	len(text) < threshold	Warning to user; prompt re-upload

Error Scenario	Detection	Response
LLM returns invalid JSON	json.JSONDecodeError	Retry up to 3 times; fallback error if all fail
LLM timeout (>120 sec)	asyncio.TimeoutError	HTTP 504 with retry suggestion
ChromaDB collection error	Exception in query	Fallback to full-text prompt (no RAG)
DOCX/PDF export failure	Exception in generation	HTTP 500 with export error message
SQLite write failure	OperationalError	Log error; generation still succeeds without saving to bank

9. Technology Stack Summary

Layer	Technology	Version	Justification
Backend Framework	FastAPI	0.110+	Async support, auto-generated API docs, Pydantic integration
LLM Runtime	Ollama	Latest	Local model serving with REST API, GPU/CPU auto-detection
Language Model	LLaMA 3 8B (q4_K_M)	Meta 2024	Best open-source instruction model, GGUF quantized for efficiency
Vector Database	ChromaDB	0.4+	Embedded mode, no external server, Python-native
Embedding Model	all-MiniLM-L6-v2	Sentence Transformers	Fast, small, accurate; ideal for RAG on CPU
PDF Extraction	PyMuPDF	1.23+	Fastest Python PDF library; native text + rendering
OCR Engine	Tesseract 5 + pytesseract	5.x	Industry standard; supports Urdu with tessdata
Image Processing	OpenCV	4.x	Preprocessing pipeline for OCR quality improvement
DOCX Export	python-docx	1.x	Full Word document generation with templates
PDF Export	WeasyPrint	60+	CSS-based PDF rendering from HTML
Database	SQLite + SQLAlchemy	Built-in	Zero-config, file-based, sufficient for question bank
Frontend	HTML/CSS/JS	Latest	Lightweight, no build tools required
Task Scheduling	APScheduler	3.x	Background jobs for session cleanup

Layer	Technology	Version	Justification
Validation	Pydantic	2.x	Runtime type checking for API and LLM outputs

10. Appendix

10.1 Folder Structure

```
exam-generator/
├── backend/
│   ├── main.py
│   ├── routers/
│   │   ├── upload.py
│   │   ├── generate.py
│   │   ├── export.py
│   │   └── bank.py
│   ├── services/
│   │   ├── ocr_service.py
│   │   ├── text_processor.py
│   │   ├── vector_store.py
│   │   ├── llm_service.py
│   │   ├── validator.py
│   │   ├── deduplicator.py
│   │   ├── bloom_classifier.py
│   │   └── export_service.py
│   ├── models/
│   │   └── schemas.py
│   ├── db/
│   │   ├── database.py
│   │   └── models.py
│   └── utils/
│       ├── prompt_templates.py
│       └── session_manager.py
├── frontend/
│   ├── src/
│   │   ├── pages/
│   │   ├── components/
│   │   └── api/
│   └── public/
├── templates/
│   ├── exam_template.docx
│   └── exam_template.html
└── requirements.txt
```

10.2 Revision History

Version	Date	Author	Changes
1.0	April 2025	Mujeeb ur Rehman & Madad Abbas	Initial SDD Document

10.3 Approval

Role	Name	Signature	Date
Student / Developer	Mujeeb ur Rehman (21886003)		
Student / Developer	Madad Abbas (21886010)		
FYP Supervisor	Syeda Asma bibi		
Department Head			

Document End — Software Design Document v1.0